AFLL PLY Tool Assignment

Name: Vivian Sobers E
SRN: PES1UG24CS901
Class: 3 'C'

Syntax Analysis using PLY

1. Language: Javascript

2. Constructs: - Variable declarations - Function definitions - Selective constructs (if/else) - Iterative constructs (for/while) - Array declarations

---

Constructs Implemented

1. **Variable Declaration/Instantiation**
   - `var`, `let`, `const` keywords
   - With and without initialization
2. **Function Definition**
   - Function declarations with parameters
   - Function body with statements
3. **Selective Construct**
   - `if` statements
   - `if-else` statements
   - Nested conditions
4. **Iterative Construct**
   - `for` loops
   - `while` loops
5. **Array Declaration**
   - Array initialization with elements
   - Empty arrays

Implementation

Lexer (lexer.py)

The lexer performs **lexical analysis** by breaking down the input JavaScript code into tokens.

```python
import ply.lex as lex

reserved = {
    'var': 'VAR',
    'let': 'LET',
    'const': 'CONST',
    'function': 'FUNCTION',
    'if': 'IF',
    'else': 'ELSE',
    'for': 'FOR',
    'while': 'WHILE',
    'return': 'RETURN',
    'true': 'TRUE',
    'false': 'FALSE'
}

tokens = [
    'IDENTIFIER', 'NUMBER', 'STRING',
    'PLUS', 'MINUS', 'TIMES', 'DIVIDE',
    'ASSIGN', 'SEMICOLON', 'COMMA',
    'LPAREN', 'RPAREN', 'LBRACE', 'RBRACE',
    'LBRACKET', 'RBRACKET',
    'GT', 'LT', 'GTE', 'LTE', 'EQEQ', 'NEQ'
] + list(reserved.values())

t_PLUS       = r'\+'
t_MINUS      = r'-'
t_TIMES      = r'\*'
t_DIVIDE     = r'/'
t_ASSIGN     = r'='
t_SEMICOLON  = r';'
t_COMMA      = r','
t_LPAREN     = r'\('
t_RPAREN     = r'\)'
t_LBRACE     = r'\{'
t_RBRACE     = r'\}'
t_LBRACKET   = r'\['
t_RBRACKET   = r'\]'
t_GT    = r'>'
t_LT    = r'<'
t_GTE   = r'>='
t_LTE   = r'<='
t_EQEQ  = r'=='
t_NEQ   = r'!='

def t_IDENTIFIER(t):
    r'[A-Za-z_][A-Za-z0-9_$]*'
    t.type = reserved.get(t.value, 'IDENTIFIER')
    return t
```

```python
def t_NUMBER(t):
    r'\d+'
    t.value = int(t.value)
    return t

def t_STRING(t):
    r'\"([^\\\n]|(\\.))*?\"'
    return t

t_ignore = ' \t'

def t_newline(t):
    r'\n+'
    t.lexer.lineno += len(t.value)

def t_error(t):
    print(f"Illegal character '{t.value[0]}' at line {t.lineno}")
    t.lexer.skip(1)

lexer = lex.lex()
```

Key Features: - Recognizes JavaScript reserved keywords - Tokenizes identifiers, numbers, strings, and operators - Handles comparison and arithmetic operators - Tracks line numbers for error reporting

---

Parser (parser.py)

The parser performs **syntax analysis** using grammar rules defined for JavaScript constructs.

```python
import ply.yacc as yacc
from lexer import tokens

had_error = False

def p_program(p):
    '''program : statement_list'''
    pass

def p_statement_list(p):
    '''statement_list : statement_list statement
                      | statement'''
    pass

def p_statement(p):
    '''statement : var_decl
                 | array_decl
```

```
                  | func_def
                  | selection
                  | iteration
                  | return_stmt
                  | assignment
                  | empty'''
    pass

def p_var_decl(p):
    '''var_decl : VAR IDENTIFIER SEMICOLON
                | VAR IDENTIFIER ASSIGN expression SEMICOLON
                | LET IDENTIFIER SEMICOLON
                | LET IDENTIFIER ASSIGN expression SEMICOLON
                | CONST IDENTIFIER ASSIGN expression SEMICOLON'''
    pass

def p_array_decl(p):
    '''array_decl : VAR IDENTIFIER ASSIGN LBRACKET array_elements RBRACKET
SEMICOLON'''
    pass

def p_array_elements(p):
    '''array_elements : array_elements COMMA expression
                      | expression
                      | empty'''
    pass

def p_func_def(p):
    '''func_def : FUNCTION IDENTIFIER LPAREN param_list RPAREN LBRACE
statement_list RBRACE'''
    pass

def p_param_list(p):
    '''param_list : param_list COMMA IDENTIFIER
                  | IDENTIFIER
                  | empty'''
    pass

def p_selection(p):
    '''selection : IF LPAREN expression RPAREN statement
                 | IF LPAREN expression RPAREN LBRACE statement_list RBRACE
                 | IF LPAREN expression RPAREN statement ELSE statement
                 | IF LPAREN expression RPAREN LBRACE statement_list RBRACE
ELSE LBRACE statement_list RBRACE
                 | IF LPAREN expression RPAREN LBRACE statement_list RBRACE
ELSE selection'''
    pass

def p_iteration(p):
```

```
    '''iteration : FOR LPAREN var_decl expression SEMICOLON assignment RPAREN
LBRACE statement_list RBRACE
                 | WHILE LPAREN expression RPAREN LBRACE statement_list
RBRACE'''
    pass

def p_return_stmt(p):
    '''return_stmt : RETURN expression SEMICOLON'''
    pass

def p_assignment(p):
    '''assignment : IDENTIFIER ASSIGN expression SEMICOLON'''
    pass

def p_expression(p):
    '''expression : expression PLUS term
                  | expression MINUS term
                  | expression GT term
                  | expression LT term
                  | expression EQEQ term
                  | expression NEQ term
                  | term'''
    pass

def p_term(p):
    '''term : NUMBER
            | IDENTIFIER
            | STRING'''
    pass

def p_empty(p):
    'empty :'
    pass

def p_error(p):
    global had_error
    had_error = True
    if p:
        print(f"Syntax error at '{p.value}' (line {p.lineno})")
    else:
        print("Syntax error at EOF")

parser = yacc.yacc()
```

Grammar Rules: - **p_var_decl**: Handles variable declarations with var, let, and const - **p_array_decl**: Handles array declarations - **p_func_def**: Handles function definitions with parameters - **p_selection**: Handles if-else statements - **p_iteration**: Handles for and while loops - **p_expression**: Handles arithmetic and logical expressions

Main Program (main.py)

The main program provides an interactive menu for testing different JavaScript constructs.

```python
from parser import parser, had_error as parser_had_error
import parser

def verify(code):
    print("\n--- Verifying Syntax ---")
    parser.had_error = False
    parser.parser.parse(code)
    if parser.had_error:
        print("   Syntax errors found.")
    else:
        print("   Syntax is valid! No errors detected.")

def code_input():
    print("\nEnter your JavaScript code below.")
    print("Type 'exit' on a new line to verify.\n")
    code = ""
    while True:
        line = input("js> ")
        if line.strip().lower() == "exit":
            break
        code += line + "\n"
    return code

def main():
    while True:
        print("JavaScript Syntax Verifier (PLY)")
        print("1. Variable declaration/instantiation")
        print("2. Iterative constructs (for/while)")
        print("3. Selective constructs (if/else)")
        print("4. Function declaration")
        print("5. Array declaration")
        print("6. Combination of all the above constructs")
        print("7. Exit")
        choice = input("\nEnter your choice: ")

        if choice == "1":
            code = code_input()
            verify(code)
        elif choice == "2":
            code = code_input()
            verify(code)
        elif choice == "3":
            code = code_input()
            verify(code)
```

```python
        elif choice == "4":
            code = code_input()
            verify(code)
        elif choice == "5":
            code = code_input()
            verify(code)
        elif choice == "6":
            code = code_input()
            verify(code)
        elif choice == "7":
            break
        else:
            print("Invalid choice. Please enter a number between 1 and 7.\n")

if __name__ == "__main__":
    main()
```

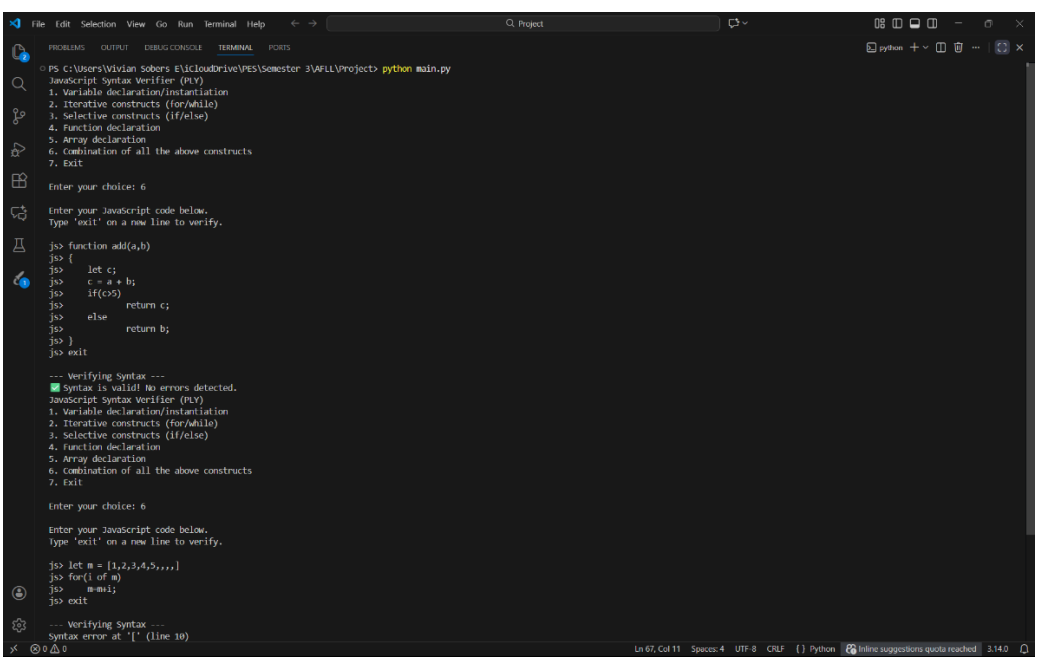
Features: - Interactive menu-driven interface - Multi-line code input - Real-time syntax verification - Clear success/error messages

---

Output Screenshots

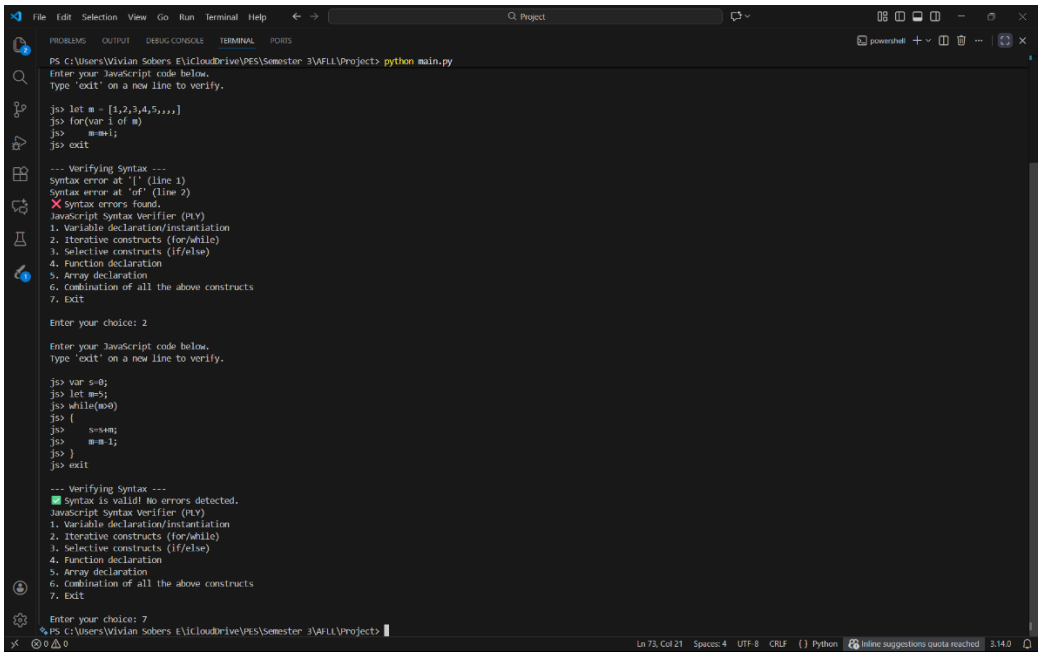
Screenshot 1: Multiple Test Cases

This screenshot demonstrates various test scenarios including error detection and successful validation.



Screenshot 2: Function and Combined Tests

This screenshot shows function definition validation and combination testing.



Test Cases

Test Case	Construct	Input	Expected Result	Actual Result
1	Array Declaration	`let m = [1,2,3,4,5,,,,]`	Syntax Error	Error Detected
2	While Loop	`while(m>0) { s=s+m; m=m-1; }`	Valid	Valid
3	Function Definition	`function add(a,b) { ... }`	Valid	Valid
4	If-Else	`if(c>5) return c; else return b;`	Valid	Valid
5	Variable Declaration	`var s=0;`	Valid	Valid

Test Case	Construct	Input	Expected Result	Actual Result
		`let m=5;`		